

EFICIENCIA DE TESTES:

EVOSUITE VS MODELOS I.A.

IMPORTANCIA DE TESTES DE SOFTWARE



Os testes de software desempenham um papel fundamental no seu desenvolvimento, garantindo qualidade e segurança. A identificação de erros e defeitos com antecedência, geram uma redução nos custos e no tempo de correção dos testes.

Para este projeto utilizamos duas ferramentas de validação de testes, o *JaCoCo* e o *Pitest*. Estas foram utilizadas para avaliar a qualidade de testes gerados automaticamente, tanto pelo plugin do *Maven Evosuite* como por testes gerados por inteligência artificial. Através dos relatórios, pudemos verificar a qualidade dos testes e a sua cobertura.

FERRAMENTAS UTILIZADAS



Jacoco

JaCoCo, ou Java Code Coverage, é uma ferramenta utilizada para medir a cobertura do código em projetos Java. Ele analisa o código-fonte e fornece relatórios detalhados sobre quais partes do código foram executadas durante os testes. Com *JaCoCo*, os desenvolvedores podem identificar áreas do código que não estão a ser testadas adequadamente, permitindo uma melhoria na qualidade dos testes e, consequentemente, na qualidade do software final.

Try Pitch



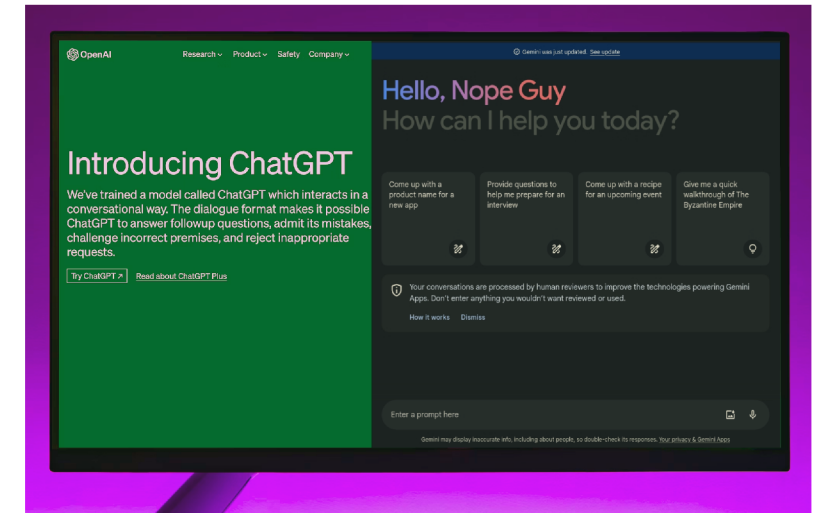
Pitest

Pitest é uma ferramenta de mutação de código para *Java* que ajuda os desenvolvedores a melhorar a eficácia dos seus testes automatizados. Ele introduz mutações (pequenas alterações) no código-fonte e executa os testes existentes para ver se consegue detetar essas mutações. Se os testes falharem ao detetar uma mutação, isso indica uma lacuna na cobertura de testes. *Pitest* então fornece informação valiosa sobre a qualidade dos testes, ajudando os desenvolvedores a escrever testes mais robustos e eficazes.



EvoSuite

Evosuite é um plugin do *Maven* que gera automaticamente casos de teste para classes *Java*. Ele analisa o código-fonte e gera testes que visam cobrir diferentes caminhos de execução do programa. Esses testes gerados automaticamente podem economizar tempo e esforço para os desenvolvedores, fornecendo uma maneira rápida de iniciar a cobertura de testes num projeto.



I.A. Modelo de Linguagem

A inteligência artificial modelo de linguagem refere-se a abordagens baseadas em *IA* que visam compreender e gerar texto em linguagem natural. Estes modelos são aplicados numa variedade de tarefas, como sumarização de texto, geração de conteúdo e muito mais. No nosso caso, foram utilizados para gerar testes unitários sobre os nossos projetos, a fim de podermos comparar a sua eficácia com a da ferramenta *Evosuite*.

UTILIZACAO DO *Maven*TM

Para iniciar este projeto, realizamos uma preparação prévia para testar os projetos *Polígono* e *Turma*. Decidimos automatizar e tornar o processo mais eficiente utilizando o *Maven*, criando um arquivo *pom.xml* com todas as especificações necessárias para os nossos objetivos, incluindo os plugins que utilizámos e as suas dependências. Embora pudéssemos ter optado pelo *IntelliJ* com plugins pré-configurados, escolhemos aplicar os conhecimentos adquiridos em aula e utilizar o *Maven* para acelerar o processo. Essa decisão permitiu-nos ter um controle mais preciso sobre as configurações do projeto e facilitou a reprodução do ambiente em diferentes sistemas, tornando mais rápido o processo de debugging.

DIFICULDADES DE TESTES: EVOSUITE

Apesar de uma vez que estivesse pronto fosse mais eficiente, configurar o projeto foi um desafio, e foi necessário tentar várias vezes para garantir que todos os plugins funcionassem corretamente ao mesmo tempo. Isso se deveu principalmente ao fato de que eles exigiam versões específicas das suas dependências, com destaque para o plugin do *EvoSuite*, cuja integração com o *JaCoCo* mostrava uma cobertura de 0% com os testes gerados. Foi então necessário consultar a documentação do *EvoSuite* para garantir que funcionava corretamente com o *JaCoCo*.

Uma vez corrigido esse problema, o *EvoSuite* foi capaz de gerar testes facilmente e nós capazes de os avaliarmos.

DIFICULDADES DE TESTES: INTELIGENCIA ARTIFICIAL

Ao contrario dos testes gerados pelo *EvoSuite*, que não apresentavam problemas no código gerado, os testes criados pelas ferramentas de inteligência artificial continham, frequentemente, erros na sua escrita.

Assim sendo, tivemos que ajustar os testes ou até mesmo adicionar novas funcionalidades no código fonte, como exceções, para garantir que os testes gerados pelas ferramentas funcionassem corretamente. Esta experiência destacou a importância da validação manual e da compreensão do contexto do projeto ao utilizar as ferramentas de geração automática de testes.

EXEMPLOS DOS TESTES GERADOS: EVOSUITE

Projeto: Poligono Triangulo_ESTest.java

```
@Test(timeout = 4000)
public void test0() throws Throwable {
    Ponto ponto0 = new Ponto();
    Ponto ponto1 = new Ponto(ponto0);
    ponto1.somaPonto((-655.022), 1238.6609295649014);
    Ponto ponto2 = new Ponto(908.1910908611, 908.1910908611);
    Triangulo triangulo0 = new Triangulo(ponto1, ponto0, ponto2);
    double double0 = triangulo0.areaTriangulo();
    assertTrue(triangulo0.fechada());
    assertEquals(859912.9827732957, double0, 0.01);
}

@Test(timeout = 4000)
public void test1() throws Throwable {
    Triangulo triangulo0 = null;
    try {
        triangulo0 = new Triangulo((Ponto) null, (Ponto) null, (Ponto) null);
        fail("Expecting exception: NullPointerException");
    } catch (NullPointerException e) {
        //
        // no message in exception (getMessage() returned null)
        //
        verifyException("poligono.Triangulo", e);
    }
}
```

Try Pitch

Projeto: Turma Aluno_ESTest.java

```
@Test(timeout = 4000)
public void test00() throws Throwable {
    HashMap<String, Double> hashMap0 = new HashMap<String, Double>();
    Double double0 = new Double((-2979.0));
    hashMap0.put("", double0);
    Aluno aluno0 = new Aluno("turma.NotaInvalidaException", "turma.NotaInvalidaException", hashMap0);
    aluno0.incrementaNota("", Notas: ", (-2979.0));
    double double1 = aluno0.media();
    assertEquals((-2979.0), double1, 0.01);
}

@Test(timeout = 4000)
public void test01() throws Throwable {
    Aluno aluno0 = new Aluno("m$e?='nVuL-`u", "m$e?='nVuL-`u");
    aluno0.setNota("/{~JR~M}[N", 20.0);
    assertEquals("m$e?='nVuL-`u", aluno0.getNome());
}

@Test(timeout = 4000)
public void test02() throws Throwable {
    HashMap<String, Double> hashMap0 = new HashMap<String, Double>();
    Aluno aluno0 = new Aluno("U<rGj3PrQq", "", hashMap0);
    aluno0.incrementaNota("", 1.0);
    double double0 = aluno0.media();
    assertEquals(1.0, double0, 0.01);
    assertEquals("U<rGj3PrQq", aluno0.getNome());
    assertEquals("", aluno0.getNumero());
}
```

RESULTADOS DO JACOCO: EVOSUITE

Testes quantitativos

Como podemos verificar cobertura grande e exemplo anterior mostra variedade em casos de teste criados com simbolos diferentes de carateres alfanumericos etc

Para ambas as classes, foi alcançada uma cobertura a 100% para as instruções , o que indica que os caminhos mais basicos, foram cobertos.

Em relação às complexidades ciclomáticas (cxy) e as linhas, para ambos os projetos - *Turma* e *Poligno* - foram cobertas na totalidade. Isto indica que todos os caminhos de fluxo de controle foram adequadamente testados.

Uma vez que não foram detetados *misses*, isto confirma que os testes criados cobrem todos os caminhos possiveis de execução, garantindo que o código tenha sido testados adequadamente.

projectturma

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
 turma		100%		100%	0	50	0	92	0	31	0	4
Total	0 of 405	100%	0 of 38	100%	0	50	0	92	0	31	0	4

turma

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
 Aluno		100%		100%	0	25	0	50	0	15	0	1
 Turma		100%		100%	0	23	0	38	0	14	0	1
 NotInvalidaException		100%		n/a	0	1	0	2	0	1	0	1
 AlunoInexistenteException		100%		n/a	0	1	0	2	0	1	0	1
Total	0 of 405	100%	0 of 38	100%	0	50	0	92	0	31	0	4

projectpoligono

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
 poligono		100%		100%	0	59	0	112	0	40	0	6
Total	0 of 683	100%	0 of 38	100%	0	59	0	112	0	40	0	6

poligono

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
 Poligono		100%		100%	0	18	0	36	0	8	0	1
 Ponto		100%		100%	0	25	0	26	0	17	0	1
 Triangulo		100%		n/a	0	4	0	17	0	4	0	1
 Retangulo		100%		n/a	0	4	0	17	0	4	0	1
 PoligonoConvexo		100%		100%	0	7	0	15	0	6	0	1
 NaoConvexoException		100%		n/a	0	1	0	1	0	1	0	1
Total	0 of 683	100%	0 of 38	100%	0	59	0	112	0	40	0	6

RESULTADOS DO PITEST: EVOSUITE

Testes qualitativos

De acordo com os resultados, os testes executados através do *EvoSuite* apresentaram resultados satisfatórios em termos de cobertura de linhas, alcançando 100% de cobertura para todas as classes, *Poligono* e *Turma*. No entanto, a cobertura de mutação, que mede a capacidade dos testes em detetar e matar modificações de código, revelou que são necessárias melhorias.

Para fortalecer ainda mais o conjunto de testes, concentrarmos nas áreas com cobertura de mutação mais baixa. De modo a melhorar os casos de teste que detectar e matar mais mutações resultaria num conjunto de testes mais robusto e confiável, aumentando a confiança na correção do software e reduzindo a probabilidade de não serem detetados alguns problemas.

Por fim, apesar da '*Line Coverage*' indicar que o conjunto de testes é abrangente, ainda há espaço para melhorias na deteção e na morte das modificações do código, como é evidente nos resultados de cobertura de mutações. Abordar estas lacunas fortalece ainda mais o conjunto de testes e contribuiu para a qualidade do software.

Pit Test Coverage Report

Package Summary

poligono

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
5	100% 111/111	89% 89/100	89% 89/100

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
Poligono.java	100% 36/36	80% 33/41	80% 33/41
PoligonoConvexo.java	100% 15/15	100% 8/8	100% 8/8
Ponto.java	100% 26/26	91% 29/32	91% 29/32
Retangulo.java	100% 17/17	100% 4/4	100% 4/4
Triangulo.java	100% 17/17	100% 15/15	100% 15/15

Report generated by [PIT](#) 1.15.2

Pit Test Coverage Report

Package Summary

turma

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
2	100% 88/88	87% 41/47	87% 41/47

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
Aluno.java	100% 50/50	91% 21/23	91% 21/23
Turma.java	100% 38/38	83% 20/24	83% 20/24

Report generated by [PIT](#) 1.15.2

Poligono.java

Mutations	
32	1. replaced return value with Collections.emptyList for poligono/Poligono::getPoligono → KILLED
46	1. Replaced integer subtraction with addition → KILLED
	2. replaced boolean return with true for poligono/Poligono::fechada → KILLED
	3. replaced boolean return with false for poligono/Poligono::fechada → KILLED
51	1. changed conditional boundary → KILLED
	2. negated conditional → KILLED
52	1. Replaced integer subtraction with addition → KILLED
	2. Replaced double addition with subtraction → KILLED
53	1. replaced double return with 0.0d for poligono/Poligono::perimetro → KILLED
69	1. negated conditional → KILLED
	2. changed conditional boundary → KILLED
70	1. replaced boolean return with false for poligono/Poligono::eConvexo → KILLED
75	1. changed conditional boundary → KILLED
	2. negated conditional → KILLED
76	1. Replaced double subtraction with addition → SURVIVED
	2. Replaced integer modulus with multiplication → KILLED
	3. Replaced integer addition with subtraction → KILLED
	4. Replaced integer addition with subtraction → KILLED
	5. Replaced integer modulus with multiplication → KILLED
77	1. Replaced integer modulus with multiplication → KILLED
	2. Replaced integer addition with subtraction → KILLED
	3. Replaced integer addition with subtraction → KILLED
	4. Replaced integer modulus with multiplication → KILLED
	5. Replaced double subtraction with addition → SURVIVED
78	1. Replaced double subtraction with addition → SURVIVED
	2. Replaced integer modulus with multiplication → KILLED
	3. Replaced integer addition with subtraction → KILLED
79	1. Replaced double subtraction with addition → SURVIVED
	2. Replaced integer modulus with multiplication → KILLED
	3. Replaced integer addition with subtraction → KILLED
80	1. Replaced double subtraction with addition → SURVIVED
	2. Replaced double multiplication with division → SURVIVED
	3. Replaced double multiplication with division → SURVIVED
81	1. negated conditional → SURVIVED
82	1. negated conditional → KILLED
	2. changed conditional boundary → KILLED
83	1. negated conditional → KILLED
	2. changed conditional boundary → KILLED
	3. negated conditional → KILLED
84	1. replaced boolean return with true for poligono/Poligono::eConvexo → KILLED
88	1. replaced boolean return with false for poligono/Poligono::eConvexo → KILLED

Turma.java

Mutations	
16	1. removed call to turma/Turma::setAlunos → KILLED
28	1. replaced return value with Collections.emptyList for turma/Turma::getAlunos → KILLED
32	1. negated conditional → KILLED
36	1. replaced boolean return with true for turma/Turma::lambda\$containsAluno\$0 → SURVIVED
	2. replaced boolean return with false for turma/Turma::lambda\$containsAluno\$0 → KILLED
	3. negated conditional → KILLED
	4. replaced boolean return with true for turma/Turma::containsAluno → KILLED
	5. changed conditional boundary → KILLED
42	1. negated conditional → KILLED
	2. negated conditional → KILLED
44	1. negated conditional → KILLED
45	1. removed call to java/util/Iterator::remove → SURVIVED
49	1. negated conditional → KILLED
54	1. replaced boolean return with true for turma/Turma::lambda\$getAluno\$1 → SURVIVED
	2. replaced return value with null for turma/Turma::getAluno → KILLED
	3. replaced return value with null for turma/Turma::lambda\$getAluno\$2 → KILLED
	4. replaced boolean return with false for turma/Turma::lambda\$getAluno\$1 → KILLED
61	1. Replaced double addition with subtraction → KILLED
62	1. Changed increment from 1 to -1 → KILLED
64	1. replaced double return with 0.0d for turma/Turma::media → KILLED
	2. Replaced double division with multiplication → KILLED
70	1. negated conditional → KILLED
	2. changed conditional boundary → KILLED
72	1. replaced return value with Collections.emptyList for turma/Turma::reprovados → SURVIVED

Try Pitch

ANALISE DETALHADA DE TESTE DE MUTACOES: EVOSUITE

Para começar, examinámos o ficheiro *Pitest*, reparámos que os testes gerados pelo *Polígono* falhavam maioritariamente quando ocorria substituições de operadores matemáticos em operações com números em vírgula flutuante. Na classe *Turma*, verificamos que as falhas eram mais frequentes em testes que necessitavam de obter valores de uma estrutura de dados.

Possivelmente, isto poderia ter sido melhorado através da utilização de mais cores ou mais tempo na geração de testes, de modo a permitir à ferramenta verificar essas ramificações não cobertas e criar testes para as mesmas. Contudo, tal não é garantido, dependendo da complexidade do código-fonte.

CONCLUSOES: EVOSUITE

Gerar testes automáticos pelo *EvoSuite* é uma ferramenta útil para identificar áreas de oportunidade de teste e garantir que o software seja robusto e confiável. No entanto, os resultados demonstram a importância de complementar os testes gerados automaticamente com testes manualmente escritos para garantir que todos os caminhos de execução sejam adequadamente testados.

Além disso, a avaliação realizada pelo *JaCoCo* e pelo *Pitest* revela que embora o *EvoSuite* seja eficaz na geração de testes que alcançam uma cobertura de instruções e ramificações satisfatória, ainda há margem para melhorias na cobertura por meio da reescrita de testes. A análise detalhada desses resultados destaca a importância de uma abordagem abrangente na validação de software, combinando tanto os testes gerados automaticamente quanto os manualmente escritos. A complementação dos testes gerados automaticamente com testes manualmente escritos é crucial para garantir que todos os caminhos de execução sejam adequadamente testados e que o software seja robusto e confiável em diversas situações e cenários de uso.

Concluindo, poderia também ser melhorada a eficácia dos testes através da escrita manual de mais testes como aumento de número de cores e tempo na geração dos testes por parte do *EvoSuite*, apesar destes mesmos já serem bastante completos.

EXEMPLOS DOS TESTES GERADOS: CHATGPT

Projeto: Poligno
TrianguloTest.java

```
package poligono;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class TrianguloTest {

    @Test
    void testAreaTriangulo() {
        Ponto p1 = new Ponto(0, 0);
        Ponto p2 = new Ponto(4, 0);
        Ponto p3 = new Ponto(0, 3);
        Triangulo triangulo = new Triangulo(p1, p2, p3);
        assertEquals(6.0, triangulo.areaTriangulo(), 0.001, "Area of triangle should be 6.0");
    }

    @Test
    void testClone() {
        Ponto p1 = new Ponto(0, 0);
        Ponto p2 = new Ponto(4, 0);
        Ponto p3 = new Ponto(0, 3);
        Triangulo original = new Triangulo(p1, p2, p3);
        Triangulo clone = original.clone();
        assertNotSame(original, clone, "Cloned object should not be the same instance");
    }
}
```

Try Pitch

Projeto: Turma
AlunoTest.java

```
@Test
void testSetNotaValida() {
    assertDoesNotThrow(() -> aluno.setNota("Matemática", 15));
    assertEquals(15, aluno.getNota("Matemática"), 0.01, "Nota deveria ser 15");
}

@Test
void testSetNotaInvalida() {
    assertThrows(NotaInvalidaException.class, () -> aluno.setNota("Matemática", 21), "Deveria lançar NotaInvalidaException");
}

@Test
void testMediaNotas() {
    assertAll(
        () -> aluno.setNota("Matemática", 14),
        () -> aluno.setNota("Português", 16)
    );
    assertEquals(15, aluno.media(), 0.01, "A média deveria ser 15");
}

@Test
void testClone() {
    Aluno clone = aluno.clone();
    assertNotSame(aluno, clone, "Clonado deve ser um novo objeto");
    assertEquals(aluno.getNome(), clone.getNome(), "Nomes devem ser iguais");
    assertEquals(aluno.getNumero(), clone.getNumero(), "Números devem ser iguais");
}
```

projectturma

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
poligono		38%		10%	38	59	59	112	21	40	1	6
Total	420 of 683	38%	34 of 38	10%	38	59	59	112	21	40	1	6

poligono

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
Poligono		22%		20%	12	18	25	36	4	8	0	1
Ponto		24%		0%	19	25	19	26	11	17	0	1
PoligonoConvexo		4%		0%	6	7	14	15	5	6	0	1
NaoConvexoException		0%		n/a	1	1	1	1	1	1	1	1
Triangulo		100%		n/a	0	4	0	17	0	4	0	1
Retangulo		100%		n/a	0	4	0	17	0	4	0	1
Total	420 of 683	38%	34 of 38	10%	38	59	59	112	21	40	1	6

projectturma

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
turma		69%		78%	20	50	27	92	12	31	0	4
Total	122 of 404	69%	8 of 38	78%	20	50	27	92	12	31	0	4

turma

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
Aluno		64%		65%	13	25	20	50	6	15	0	1
Turma		75%		94%	7	23	7	38	6	14	0	1
NotaInvalidaException		100%		n/a	0	1	0	2	0	1	0	1
AlunoInexistenteException		100%		n/a	0	1	0	2	0	1	0	1
Total	122 of 404	69%	8 of 38	78%	20	50	27	92	12	31	0	4

RESULTADOS DO JACOCO: CHATGPT

Testes quantitativos

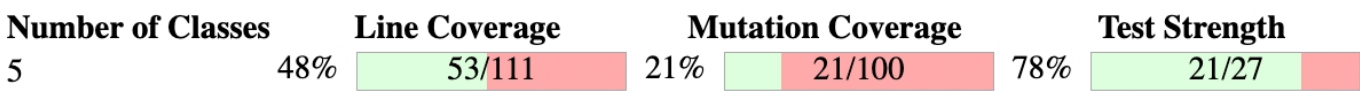
A análise dos resultados dos testes de cobertura gerados pelo *ChatGPT* usando o *Jacoco*, mostra que para a classe *Poligno*, apenas 38% de todas as instruções no código foram executadas pelos testes e que apenas 10% de todos os branches no código foram executados pelos testes. Em relação ao número total de instruções e ao número total de branches, estes números indicam que a cobertura de testes foi baixa, o que levou a bugs que não foram detectados.

Em relação à cobertura dos testes da classe *Turma*, esta tem uma cobertura de testes significativamente melhor do que a classe anterior em ambas as métricas: cobertura de instruções e cobertura de branches. Isto sugere que há uma maior confiabilidade nos testes e uma cobertura de código mais completa.

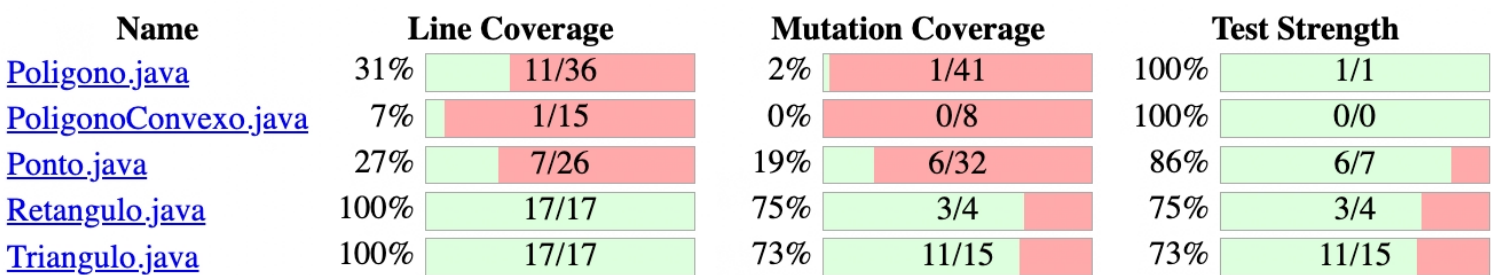
Pit Test Coverage Report

Package Summary

poligono



Breakdown by Class



RESULTADOS DO PITEST: CHATGPT

Testes qualitativos

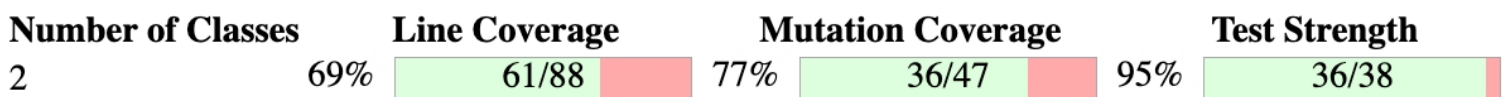
De um modo geral, os resultados do *Pitest* mostraram uma cobertura de mutação mais baixa e testes menos eficazes em comparação com os testes individuais gerados pelas outras ferramentas usadas previamente.

Além disso, os testes da classe *Poligno* foram relativamente piores do que os da classe *Turma*. Os resultados do *Pitest* demonstram que, apesar do *GPT* ser uma ferramenta bastante útil, ainda desenvolve material que muitas vezes não está 100% de acordo com o que lhe foi pedido e não tem a capacidade de ter uma noção global dos projetos para cobrir a maioria dos casos necessários para testes unitários, principalmente em casos de projetos que necessitam de uma maior lógica matemática, como visto no *Polígono*.

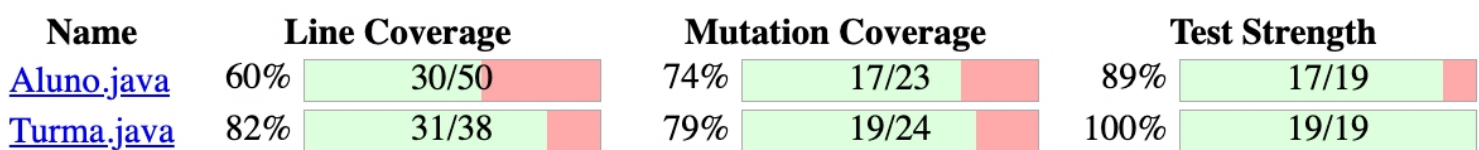
Pit Test Coverage Report

Package Summary

turma



Breakdown by Class



Poligono.java

Mutations	
32	1. replaced return value with Collections.emptyList for poligono/Poligono::getPoligono → KILLED
46	1. Replaced integer subtraction with addition → NO_COVERAGE 2. replaced boolean return with true for poligono/Poligono::fechada → NO_COVERAGE 3. replaced boolean return with false for poligono/Poligono::fechada → NO_COVERAGE
51	1. changed conditional boundary → NO_COVERAGE 2. negated conditional → NO_COVERAGE
52	1. Replaced integer subtraction with addition → NO_COVERAGE 2. Replaced double addition with subtraction → NO_COVERAGE
53	1. replaced double return with 0.0d for poligono/Poligono::perimetro → NO_COVERAGE
69	1. negated conditional → NO_COVERAGE 2. changed conditional boundary → NO_COVERAGE
70	1. replaced boolean return with false for poligono/Poligono::eConvexo → NO_COVERAGE
75	1. changed conditional boundary → NO_COVERAGE 2. negated conditional → NO_COVERAGE
76	1. Replaced double subtraction with addition → NO_COVERAGE 2. Replaced integer modulus with multiplication → NO_COVERAGE 3. Replaced integer addition with subtraction → NO_COVERAGE 4. Replaced integer addition with subtraction → NO_COVERAGE 5. Replaced integer modulus with multiplication → NO_COVERAGE
77	1. Replaced integer modulus with multiplication → NO_COVERAGE 2. Replaced integer addition with subtraction → NO_COVERAGE 3. Replaced integer addition with subtraction → NO_COVERAGE 4. Replaced integer modulus with multiplication → NO_COVERAGE 5. Replaced double subtraction with addition → NO_COVERAGE
78	1. Replaced double subtraction with addition → NO_COVERAGE 2. Replaced integer modulus with multiplication → NO_COVERAGE 3. Replaced integer addition with subtraction → NO_COVERAGE
79	1. Replaced double subtraction with addition → NO_COVERAGE 2. Replaced integer modulus with multiplication → NO_COVERAGE 3. Replaced integer addition with subtraction → NO_COVERAGE
80	1. Replaced double subtraction with addition → NO_COVERAGE 2. Replaced double multiplication with division → NO_COVERAGE 3. Replaced double multiplication with division → NO_COVERAGE
81	1. negated conditional → NO_COVERAGE
82	1. negated conditional → NO_COVERAGE 2. changed conditional boundary → NO_COVERAGE
83	1. negated conditional → NO_COVERAGE 2. changed conditional boundary → NO_COVERAGE 3. negated conditional → NO_COVERAGE
84	1. replaced boolean return with true for poligono/Poligono::eConvexo → NO_COVERAGE
88	1. replaced boolean return with false for poligono/Poligono::eConvexo → NO_COVERAGE

Turma.java

Mutations	
16	1. removed call to turma/Turma::setAlunos → KILLED
28	1. replaced return value with Collections.emptyList for turma/Turma::getAlunos → NO_COVERAGE
32	1. negated conditional → KILLED
36	1. replaced boolean return with true for turma/Turma::lambda\$containsAluno\$0 → KILLED 2. replaced boolean return with false for turma/Turma::lambda\$containsAluno\$0 → KILLED 3. negated conditional → KILLED 4. replaced boolean return with true for turma/Turma::containsAluno → KILLED 5. changed conditional boundary → KILLED
42	1. negated conditional → KILLED 2. negated conditional → KILLED
44	1. negated conditional → KILLED
45	1. removed call to java/util/Iterator::remove → KILLED
49	1. negated conditional → KILLED
54	1. replaced boolean return with true for turma/Turma::lambda\$getAluno\$1 → NO_COVERAGE 2. replaced return value with null for turma/Turma::getAluno → NO_COVERAGE 3. replaced return value with null for turma/Turma::lambda\$getAluno\$2 → NO_COVERAGE 4. replaced boolean return with false for turma/Turma::lambda\$getAluno\$1 → NO_COVERAGE
61	1. Replaced double addition with subtraction → KILLED
62	1. Changed increment from 1 to -1 → KILLED
64	1. replaced double return with 0.0d for turma/Turma::media → KILLED 2. Replaced double division with multiplication → KILLED
70	1. negated conditional → KILLED 2. changed conditional boundary → KILLED
72	1. replaced return value with Collections.emptyList for turma/Turma::reprovados → KILLED

Try Pitch

ANALISE DETALHADA DE TESTE DE MUTACOES: CHATGPT

Podemos observar que, além da cobertura ter sido aquém do esperado, a qualidade dos testes também deixa bastante a desejar, falhando em várias frentes, como na substituição de operadores matemáticos e booleanos nas classes do projeto *Polígono*.

No entanto, nas classes da *Turma*, parece não ter havido tanta dificuldade em criar testes que cobrissem esses casos, como o de negar as condicionais ou substituir valores booleanos apesar de ainda assim haver alguns erros. Isso pode ser atribuído à menor complexidade envolvida no projeto em termos matemáticos ou simplesmente devido à natureza aleatória das respostas prévias dadas pelo *GPT*.

CONCLUSOES: CHAT GPT

Apesar de ser capaz de produzir código de melhor qualidade do que outras inteligências artificiais, tornando assim a sua implementação mais fácil e breve, é importante notar que os testes gerados não são tão ótimos quanto os do *EvoSuite*. Embora sejam satisfatórios, muitas mudanças são necessárias por parte de quem os utiliza para garantir uma maior qualidade e abranger uma gama mais ampla de casos em que o código pode não funcionar corretamente.

Para além disso, sem acesso às versões pagas da ferramenta, torna-se difícil pedir testes que abranjam mais amplamente o projeto devido ao facto de não ser possível dar uma ideia completa do mesmo, e com o limite de caracteres por mensagem, ele vai se "esquecendo" de certos pormenores do projeto, fazendo testes mais específicos a cada classe, não tendo em conta o contexto global do projeto.

EXEMPLOS DOS TESTES GERADOS:

GEMINI

Testes dados

Projeto: Poligno
TrianguloTest.java

```
package poligono;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

public class TrianguloTest {

    @Test
    void testAreaTriangulo() {
        Triangulo t = new Triangulo(new Ponto(0, 0), new Ponto(1, 0),
            new Ponto(0, 1));
        assertEquals(0.5, t.areaTriangulo());
    }

    @Test
    void testClone() {
        Ponto p1 = new Ponto(0, 0);
        Ponto p2 = new Ponto(1, 0);
        Ponto p3 = new Ponto(0, 1);
        Triangulo t1 = new Triangulo(p1, p2, p3);
        Triangulo t2 = new Triangulo(p1.clone(), p2.clone(), p3.clone());

        assertNotSame(t1, t2);
        assertEquals(t1.getPoligono(), t2.getPoligono());
    }
}
```

Try Pitch

Projeto: Turma
AlunoTest.java

```
@Test
public void testSetNotaValida() {
    Aluno aluno = new Aluno("João", "123456789");
    aluno.setNota("Matemática", 10.0);

    assertEquals(10.0, aluno.getNota("Matemática"));
}

@Test
public void testSetNotaInvalidaMenorQueZero() {
    Aluno aluno = new Aluno("João", "123456789");

    assertThrows(NotaInvalidaException.class, () -> {
        aluno.setNota("Matemática", -1.0);
    });
}

@Test
public void testSetNotaInvalidaMaiorQueVinte() {
    Aluno aluno = new Aluno("João", "123456789");

    assertThrows(NotaInvalidaException.class, () -> {
        aluno.setNota("Matemática", 21.0);
    });
}
```

EXEMPLOS DOS TESTES GERADOS:

GEMINI

Testes corrigidos

Projeto: Poligno
TrianguloTest.java

```
package poligono;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

public class TrianguloTest {

    @Test
    void testAreaTriangulo() {
        Triangulo t = new Triangulo(new Ponto(0, 0), new Ponto(1, 0),
            new Ponto(0, 1));
        assertEquals(0.5, t.areaTriangulo());
    }

    @Test
    void testClone() {
        Ponto p1 = new Ponto(0, 0);
        Ponto p2 = new Ponto(1, 0);
        Ponto p3 = new Ponto(0, 1);
        Triangulo t1 = new Triangulo(p1, p2, p3);
        Triangulo t2 = new Triangulo(p1.clone(), p2.clone(), p3.clone());

        assertNotSame(t1, t2);
        assertEquals(t1.clone(), t2.clone());
    }
}
```

Try Pitch

Projeto: Turma
AlunoTest.java

```
@Test
public void testSetNotaValida() throws NotaInvalidaException {
    Aluno aluno = new Aluno("João", "123456789");
    aluno.setNota("Matemática", 10.0);

    assertEquals(10.0, aluno.getNota("Matemática"));
}

@Test
public void testSetNotaInvalidaMenorQueZero() {
    Aluno aluno = new Aluno("João", "123456789");

    assertThrows(NotaInvalidaException.class, () -> {
        aluno.setNota("Matemática", -1.0);
    });
}

@Test
public void testSetNotaInvalidaMaiorQueVinte() {
    Aluno aluno = new Aluno("João", "123456789");

    assertThrows(NotaInvalidaException.class, () -> {
        aluno.setNota("Matemática", 21.0);
    });
}
```

CORRECOES DOS TESTES GERADOS: GEMINI

Em geral, o Gemini criou testes com falhas em diversas áreas, tanto em programação base de conceitos de programação orientada a objetos, como na área de testes unitários em java (*Junit5*).

Projeto Turma

Nos testes da classe *Aluno*, o Gemini gerou testes com diversas falhas, como lançamento de exceções não pretendidas (*IllegalArgumentException*), assim como não analisou bem o *source code*, tentando usar métodos inválidos como *aluno.getNotas()*, em vez de *aluno.getNota()*, assim como a falta de inclusão de bibliotecas essenciais, como o *Java.util.**.

Na classe *Turma*, falhou novamente, de forma mais leviana nos *Imports*, nos construtores da classe, tentando invocá-los com *null*, em vez de um objeto de lista de tipo *Aluno*, mas de forma geral, falhou menos vezes nesta classe.

Projeto Poligono

No projeto do *Poligono*, o gemini inicialmente criou um *PoligonoTest.java*, tentando de maneira direta, testar a classe abstrata *Poligono*, não sendo um teste válido. Também no testes *Junit* da classe *PoligonoConvexo*, o Gemini teve diversas inconsistências na geração dos testes, como tentar utilizar métodos sem objetos (necessários).

Para além disso, não foi capaz de gerar testes em métodos onde eram utilizados unidades *float*, levando a erros de arredondamento, como por exemplo em um teste do cálculo da área de um poligono.

projectturma

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
poligono		58%		55%	30	59	48	112	18	40	1	6
Total	281 of 683	58%	17 of 38	55%	30	59	48	112	18	40	1	6

poligono

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
Ponto		44%		37%	17	25	14	26	9	17	0	1
Poligono		63%		65%	8	18	15	36	4	8	0	1
Retangulo		0%		n/a	4	4	17	17	4	4	1	1
PoligonoConvexo		94%		100%	1	7	2	15	1	6	0	1
Triangulo		100%		n/a	0	4	0	17	0	4	0	1
NaoConvexoException		100%		n/a	0	1	0	1	0	1	0	1
Total	281 of 683	58%	17 of 38	55%	30	59	48	112	18	40	1	6

projectturma

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
turma		63%		65%	27	61	37	108	11	31	1	4
Total	177 of 485	63%	21 of 60	65%	27	61	37	108	11	31	1	4

turma

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
Aluno		62%		66%	16	33	22	60	6	15	0	1
Turma		65%		62%	10	26	13	44	4	14	0	1
AlunoInexistenteException		0%		n/a	1	1	2	2	1	1	1	1
NotaInvalidaException		100%		n/a	0	1	0	2	0	1	0	1
Total	177 of 485	63%	21 of 60	65%	27	61	37	108	11	31	1	4

RESULTADOS DO JACOACO: GEMINI

Testes quantitativos

Os testes das classes *Poligno* e *Turma* apresentam diferentes padrões de cobertura de instruções e branches ausentes. Relativamente à cobertura dos testes, a classe *Turma* é mais consistente, já a classe *Poligno*, mostra uma variação mais significativa, relativamente à cobertura.

Isto implica a necessidade de testes mais detalhados para ambas as classes, de modo a garantir uma cobertura mais abrangente e consistente. Apesar disto tudo podemos aferir que os testes gerados pelo *Gemini* apresentam uma maior cobertura do que os gerados pelo *ChatGPT*.

RESULTADOS DO PITEST: GEMINI

Testes qualitativos

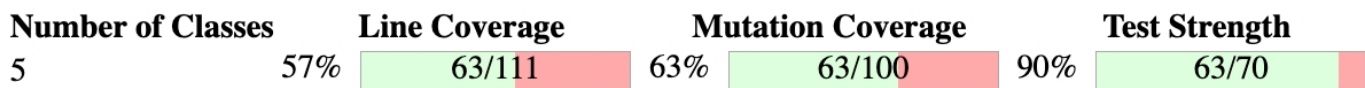
Relativamente aos testes unitários do Gemini, o *Pit* apresentou uma cobertura de testes razoável de cerca de 60% em ambos os projetos depois de feitas as correções necessárias. Apesar disso, é notável as falhas como por exemplo na classe *Retangulo*, do projeto *Poligono*.

A cobertura de mutações também foi medíocre, o que levou a um teste de nível menor, influenciando certamente os resultados obtidos, em ambos os projetos.

Pit Test Coverage Report

Package Summary

poligono



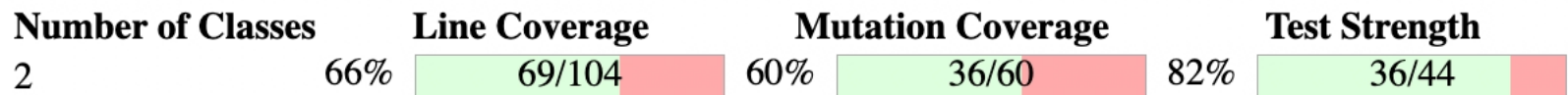
Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
Poligono.java	58% 21/36	76% 31/41	97% 31/32
PoligonoConvexo.java	87% 13/15	88% 7/8	88% 7/8
Ponto.java	46% 12/26	38% 12/32	80% 12/15
Retangulo.java	0% 0/17	0% 0/4	100% 0/0
Triangulo.java	100% 17/17	87% 13/15	87% 13/15

Pit Test Coverage Report

Package Summary

turma



Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
Aluno.java	63% 38/60	56% 18/32	75% 18/24
Turma.java	70% 31/44	64% 18/28	90% 18/20

Poligono.java

Mutations	
32	1. replaced return value with Collections.emptyList for poligono/Poligono::getPoligono → KILLED
46	1. Replaced integer subtraction with addition → NO_COVERAGE 2. replaced boolean return with true for poligono/Poligono::fechada → NO_COVERAGE 3. replaced boolean return with false for poligono/Poligono::fechada → NO_COVERAGE
51	1. changed conditional boundary → NO_COVERAGE 2. negated conditional → NO_COVERAGE
52	1. Replaced integer subtraction with addition → NO_COVERAGE 2. Replaced double addition with subtraction → NO_COVERAGE
53	1. replaced double return with 0.0d for poligono/Poligono::perimetro → NO_COVERAGE
69	1. negated conditional → KILLED 2. changed conditional boundary → SURVIVED
70	1. replaced boolean return with false for poligono/Poligono::eConvexo → NO_COVERAGE
75	1. changed conditional boundary → KILLED 2. negated conditional → KILLED
76	1. Replaced double subtraction with addition → KILLED 2. Replaced integer modulus with multiplication → KILLED 3. Replaced integer addition with subtraction → KILLED 4. Replaced integer addition with subtraction → KILLED 5. Replaced integer modulus with multiplication → KILLED
77	1. Replaced integer modulus with multiplication → KILLED 2. Replaced integer addition with subtraction → KILLED 3. Replaced integer addition with subtraction → KILLED 4. Replaced integer modulus with multiplication → KILLED 5. Replaced double subtraction with addition → KILLED
78	1. Replaced double subtraction with addition → KILLED 2. Replaced integer modulus with multiplication → KILLED 3. Replaced integer addition with subtraction → KILLED
79	1. Replaced double subtraction with addition → KILLED 2. Replaced integer modulus with multiplication → KILLED 3. Replaced integer addition with subtraction → KILLED
80	1. Replaced double subtraction with addition → KILLED 2. Replaced double multiplication with division → KILLED 3. Replaced double multiplication with division → KILLED
81	1. negated conditional → KILLED
82	1. negated conditional → KILLED 2. changed conditional boundary → KILLED
83	1. negated conditional → KILLED 2. changed conditional boundary → KILLED 3. negated conditional → KILLED
84	1. replaced boolean return with true for poligono/Poligono::eConvexo → KILLED
88	1. replaced boolean return with false for poligono/Poligono::eConvexo → KILLED

Turma.java

utations	
.	negated conditional → KILLED
.	removed call to turma/Turma::setAlunos → KILLED
.	negated conditional → KILLED
.	replaced return value with Collections.emptyList for turma/Turma::getAlunos → KILLED
.	negated conditional → KILLED
.	negated conditional → KILLED
.	negated conditional → KILLED
.	replaced boolean return with true for turma/Turma::containsAluno → KILLED
.	replaced boolean return with false for turma/Turma::containsAluno → KILLED
.	replaced boolean return with false for turma/Turma::lambda\$2 → KILLED
.	replaced boolean return with true for turma/Turma::lambda\$2 → SURVIVED
.	changed conditional boundary → KILLED
.	negated conditional → KILLED
.	negated conditional → KILLED
.	negated conditional → KILLED
.	removed call to java/util/Iterator::remove → KILLED
.	negated conditional → KILLED
.	replaced boolean return with false for turma/Turma::lambda\$3 → KILLED
.	replaced boolean return with true for turma/Turma::lambda\$3 → SURVIVED
.	replaced return value with null for turma/Turma::getAluno → KILLED
.	replaced return value with null for turma/Turma::lambda\$4 → NO_COVERAGE
.	Replaced double addition with subtraction → NO_COVERAGE
.	Changed increment from 1 to -1 → NO_COVERAGE
.	replaced double return with 0.0d for turma/Turma::media → NO_COVERAGE
.	Replaced double division with multiplication → NO_COVERAGE
.	changed conditional boundary → NO_COVERAGE
Try Pitch	conditional → NO_COVERAGE
.	replaced return value with Collections.emptyList for turma/Turma::reprovados → NO_COV

ANALISE DETALHADA DE TESTE DE MUTACOES: GEMINI

De forma geral, o gemini não demonstrou a capacidade de criar testes de forma a prevenir alterações diretas em *returns* de booleanos, assim como em funções que utilizam *doubles*. Também é notável alguma falha no que diz respeito à alteração da lógica de certos métodos, o que levou à sobrevivência de algumas mutações, como a negação de certas condicionais.

Numa análise mais detalhada, quanto ao projeto *Turma*, também se verificaram mutações que sobreviveram na chamada de construtores, assim como na alteração na incrementação de ciclos em métodos.

Quanto ao *Poligono*, não se destacaram mutações comparativamente ao projeto anterior.

CONCLUSOES: GEMINI

Analisando isoladamente o *Gemini*, é notável a falta de certos conhecimentos na área de programação orientada a objetos, e a falta de conhecimento na ferramenta de testes unitários *Junit*, como por exemplo a não utilização de `@beforeeach`, levando ao setup dos testes em cada método.

Comparativamente ao *GPT*, revelou-se uma ferramenta menos eficaz, quer a nível do *Junit*, quer de *Java*. Uma vez que foram necessárias mais iterações para chegar a um resultado razoável. Também de notar que o Gemini recusou diversas vezes na geração de testes comparada ao *GPT*. Apesar de tal, uma vez corrigidos, os testes revelaram-se ligeiramente superiores aos gerados pelo *ChatGPT* em termos de cobertura e qualidade.

Através da utilização desta ferramenta também foi de realçar o nível precário que a I.A. ainda se encontra.

O EVOSUITE SUPERA O CHATGPT E
O GEMINI EM QUALIDADE E
QUANTIDADE DE COBERTURA
NOS TESTES, MAS POR QUÊ?

RAZAO DA SUPERIORIDADE DO EVOSUITE

O *Evosuite* é uma ferramenta altamente especializada na geração automatizada de testes, especificamente projetada para maximizar a cobertura de código e identificar eventuais falhas nos programas. A sua abordagem utiliza algoritmos avançados que permite explorar diferentes combinações, resultando numa cobertura de testes superior.

Por sua vez, o *ChatGPT* é um modelo de linguagem que não está projetado para gerar testes automatizados. Apesar de poder gerar respostas e códigos, o mesmo não tem capacidade para gerar testes tão eficientes quanto as ferramentas especializadas, como é o caso do *Evosuite*.

O *Gemini* sendo também um modelo de linguagem não gerou bons testes, e visto que é uma inteligência artificial mais inexperiente que o *GPT*, cria testes com ainda mais falhas no seu código e baixa cobertura.

Em suma, o *Evosuite* supera o *ChatGPT* e o *Gemini* em qualidade e quantidade da cobertura de testes devido à abordagem sofisticada e eficiente na geração automatizada de testes. Como as ferramentas de inteligência artificial não foram especificamente criadas para gerar testes automatizados ou para maximizar a cobertura do código, isto faz com que o *Evosuite* supere as os modelos de linguagem em relação à criação de bons testes

unitários.

CONCLUSÃO

Com o foco na produção de testes unitários em *Junit*, foi notável as diferenças entre as diferentes ferramentas que temos atualmente à nossa disposição.

Das três ferramentas que analisámos, é claramente observável, tanto de maneira quantitativa como qualitativa o destaque que o *EvoSuite* apresenta em comparação aos modelos de *IA* utilizados por causa da especialização desta ferramenta para a tarefa proposta.

Dentro destes dois, também foi clara a diferença, tendo como opinião, que o *GPT* demonstrou superioridade em relação ao *Gemini* devido à sua maior "experiência".

Focando no percurso do projeto, achamos que poderíamos ter beneficiado na utilização de diferentes definições do *EvoSuite*, assim como a utilização de diversas versões do *GPT/Gemini*, de forma a termos uma análise mais aprofundada do tema em questão.

